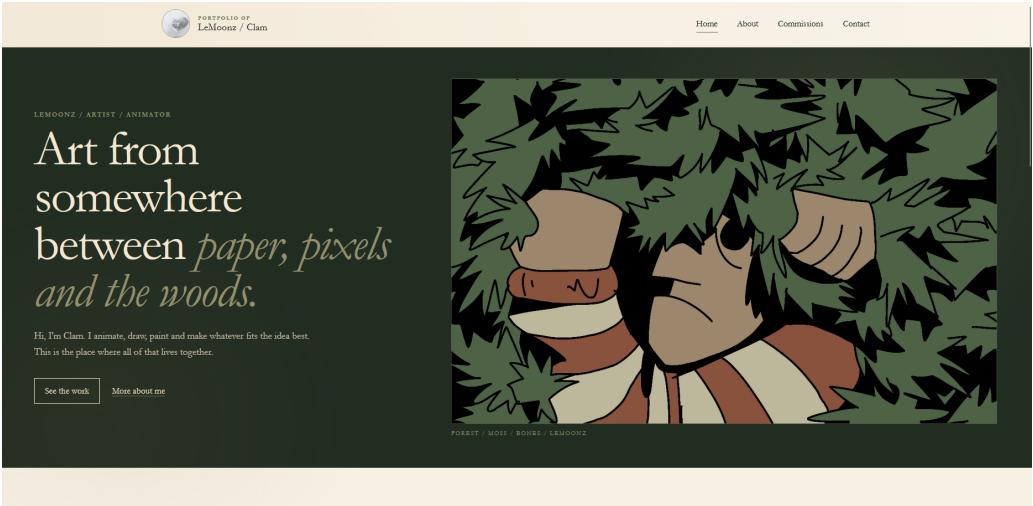


LeMoonz Portfolio

Technical documentation

Portfolio and commission information website for an independent artist and animator.

Status	Complete
Current stack	React, TypeScript, HTML, CSS, Vite
Backend	Not used
Previous prototype	Node.js, Express, EJS, JavaScript
Deployment	Railway · Docker · Caddy
Version control	Git / GitHub
Developer	Daniel Abramek
Active development	Roughly three weeks of active work
Finished	September 2026



The final version is intentionally smaller than the first technical idea. The site ended up needing a strong frontend, not a backend for the sake of having one.

Contents

Technical overview of the LeMoonz portfolio: requirements, iteration history, architecture, reusable UI, responsive design and production deployment.

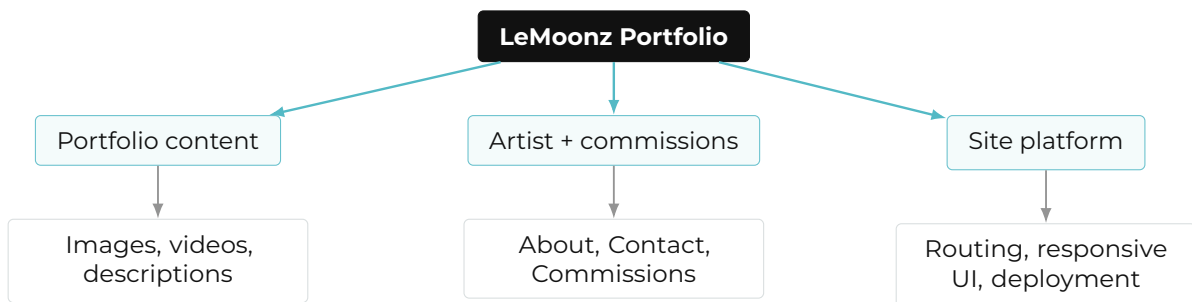
01	Project overview	1
02	Contents	2
03	Project requirements	3
04	Design and implementation iterations	4
05	EJS / Express prototype	5
06	Current frontend architecture	6
07	Routing and reusable portfolio system	7
08	Interface and responsive implementation	8
09	Production build and deployment	9
10	Final state and future development	10

Project requirements

LeMoonz is a portfolio and commission-information website built around four different kinds of artistic work: animation, digital art, traditional art and experimental pieces.

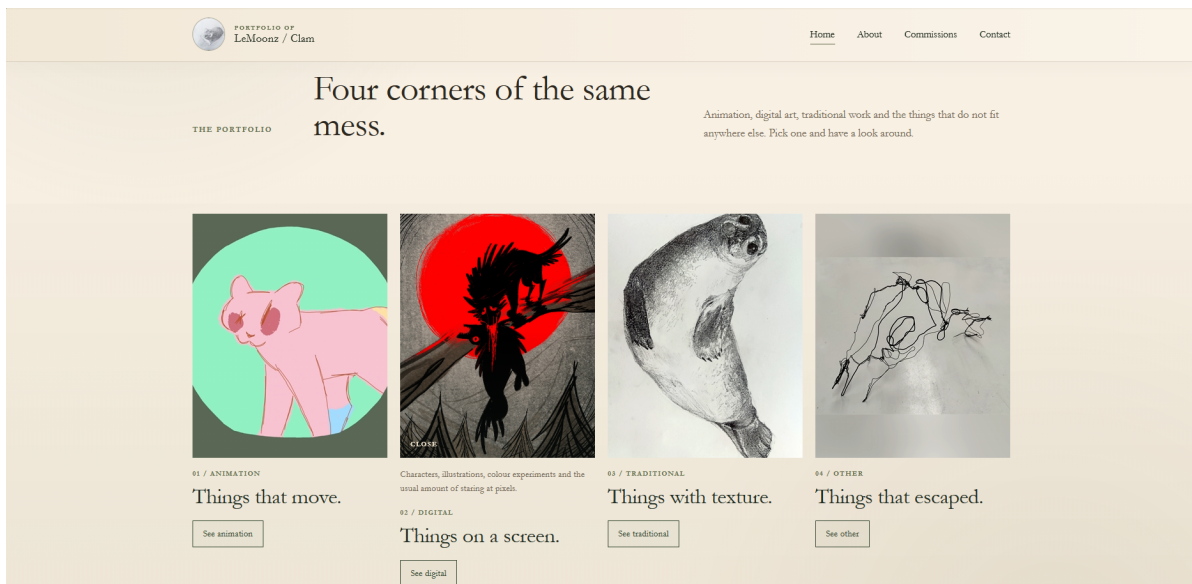
The main requirement was to keep those categories visually connected while allowing each of them to handle different media. Animation pages need video playback, image-based categories need large artwork previews, and individual pieces need enough space for titles, labels and descriptions.

The public structure also needed to stay small and predictable. The production site contains Home, About, Commissions and Contact pages, four portfolio category pages, legal pages and a custom 404 route. No authentication, dashboard or database-backed content management is required in the current version.



Functional scope

The production version is intentionally frontend-only. Its responsibilities are navigation, presentation of media, responsive layout, portfolio categorisation, commission information and static legal/contact content.



Home page category section: four portfolio entry points backed by dedicated React routes.

Design and implementation iterations

The project went through four substantial versions during roughly three weeks of active development. Each iteration changed either the content structure, visual system or implementation approach.

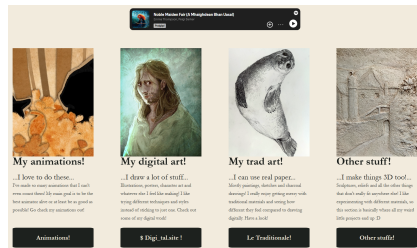
The earliest layout established the basic information hierarchy: artist introduction, work categories, commission information and contact. It was useful as a content map, but it did not yet provide a reusable visual system for a larger portfolio.

The next iterations introduced the four-category structure, a forest-inspired palette, stronger typography and larger media areas. Real artwork exposed practical layout problems that placeholders did not: inconsistent aspect ratios, long descriptions, video content and sections that needed to remain readable at very different viewport widths.

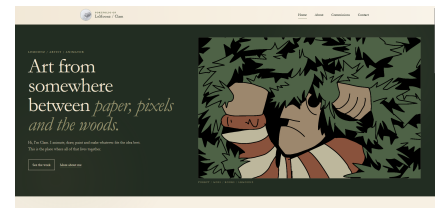
The final iteration kept the useful content structure but rebuilt the interface around reusable React components and scoped CSS. Repeated portfolio layouts became component-driven, navigation moved to React Router and the old server-rendered prototype was removed from production.



Initial structural mock-up.



Intermediate four-category layout.



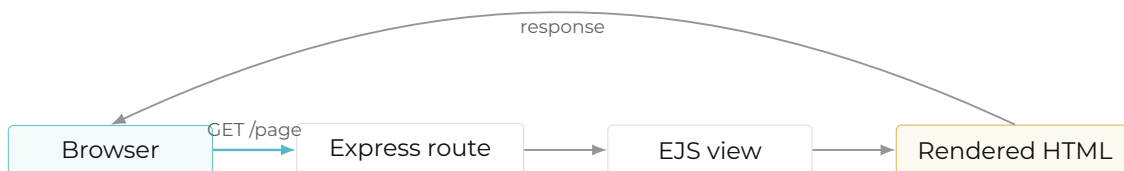
Current React production interface.

Iteration result

The four versions were not separate products. They were successive implementations of the same portfolio, with each version reducing layout problems and making the final component structure clearer.

The first technical prototype

The first working implementation used Node.js, Express, EJS and JavaScript. This prototype established the initial routing, shared page structure and deployment flow before the project moved to the current frontend-only architecture.



Each page request was matched by Express, rendered through EJS and returned as complete HTML. For an early prototype this was useful because it gave the client a real site to react to, proved the page structure and made shared elements possible through EJS partials.

The prototype used server-side rendering for pages whose final requirements were almost entirely presentation, navigation and media. Because the project had no accounts, database, dashboard or server-side business rules, Express became unnecessary in the production architecture.

Prototype outcome

The prototype validated the route structure, content hierarchy and shared layout. After those requirements stabilised, the same responsibilities could be handled by the React frontend without maintaining an application server.

What it proved

What it exposed

What survived

Page structure, early routing, content hierarchy, shared layout ideas, deployment basics.

The backend was mostly preparing pages that could live entirely in the frontend.

Content structure, visual ideas, route names and the four portfolio categories.

Current frontend architecture

The finished version is a React + TypeScript frontend built with Vite. The structure is deliberately simple: pages are pages, reusable pieces are components, styles are split by scope, and static media lives under public assets.

```
src/
|- App.tsx
|- main.tsx
|- components/
|  |- Header.tsx
|  |- Footer.tsx
|  |- PortfolioPage.tsx
|  |- PortfolioGalleryPiece.tsx
|  |- PortfolioButtons.tsx
|  |- PortfolioVine.tsx
|  |- PortfolioVineReverse.tsx
|- pages/
|  |- Home.tsx
|  |- About.tsx
|  |- Commissions.tsx
|  |- Contact.tsx
|  |- Animation.tsx
|  |- Digital.tsx
|  |- Traditional.tsx
|  |- Other.tsx
|  |- Terms.tsx
|  |- Privacy.tsx
|  |- NotFound.tsx
|- styles/
|  |- shared.css
|  |- pages/
|  |- components/
```

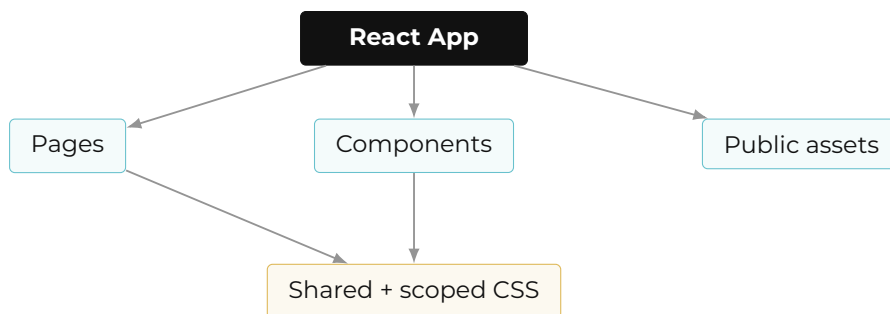
The folder structure follows the way the site is actually thought about.

Full views such as Home, About or Animation live in pages. Shared interface pieces live in components. Global visual rules stay in `shared.css`; page-specific and component-specific CSS stay closer to the thing they style.

The media structure follows the same idea. Portfolio assets are grouped by Animation, Digital, Traditional and Other instead of being dropped into one giant folder.

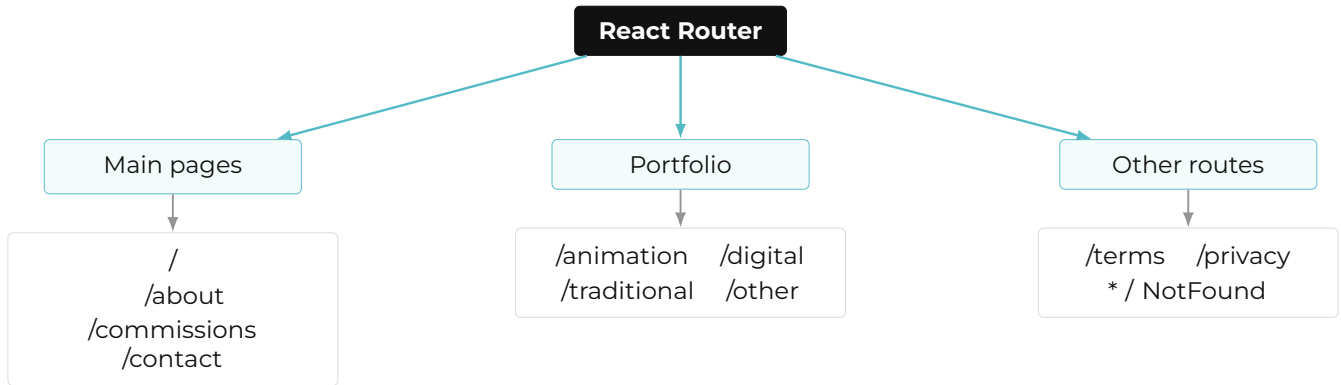
Why this structure works

The separation keeps page views, reusable UI, static assets and styles easy to locate. Changes can usually be made within one scope without searching through unrelated files.

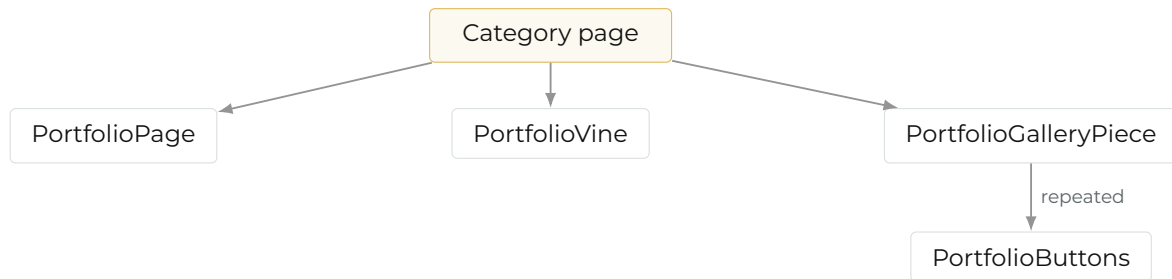


Routing and reusable portfolio system

React stayed because it solved two real problems cleanly: page-level routing and repeated portfolio UI. The four portfolio categories share structure, so that structure became reusable components instead of four copies of the same markup.



Routes are grouped here only for readability. In the actual React Router configuration they are sibling routes.

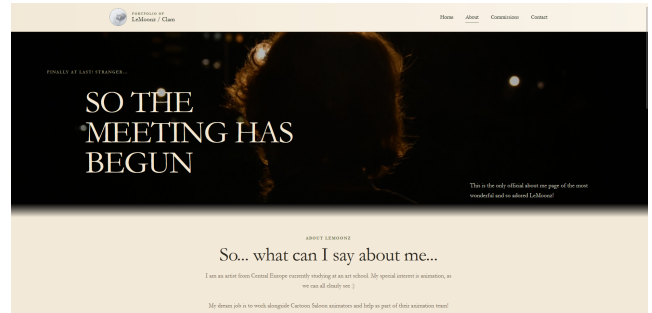
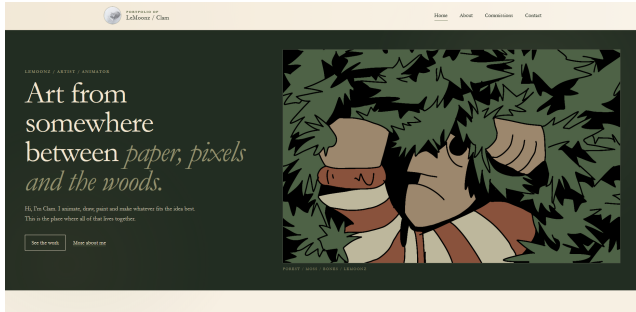


Typed media handling

`PortfolioGalleryPiece` limits `mediaType` to `'image' | 'video'`. That is a small TypeScript feature, but a useful one: the component knows exactly which markup to render. Images get lazy loading and an alt fallback; videos use muted looping playback with `playsInline`.

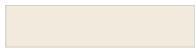
Interface, visual system and responsiveness

The final interface moved away from a normal portfolio-template look and settled into a warm forest/editorial style. The design is image-led, serif-heavy and intentionally more dramatic on desktop than on mobile.



The shared CSS owns the broad system: paper and forest colours, typography, spacing, header/footer behaviour, buttons, forms and mobile navigation. Page styles then handle the compositions that only make sense in one place, such as the Home hero or the portfolio strip.

Responsiveness is handled directly in CSS. Around 900px, large two-column compositions begin stacking. Below 760px, the global header changes to a vertical mobile layout. Around 600px, portfolio cards move to a single column and media ratios become more mobile-friendly.



Paper / main background



Forest green / primary accent



Charcoal / text and contrast



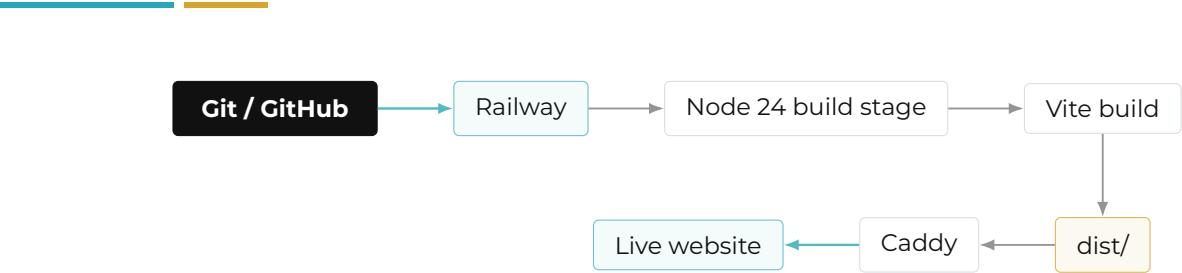
Muted olive / secondary accent

Small details that mattered

Portfolio preview buttons have visible focus states. Decorative SVG vine text is hidden from screen readers with `aria-hidden`. Image galleries use lazy loading. Hover-only hints become visible on small screens where hover does not exist.

Production build and deployment

The final site has no application backend, but it still has a production pipeline. Vite builds the frontend, Docker packages it, Caddy serves it, Railway runs it, and Git/GitHub provide version control and deployment history.



The Dockerfile uses a multi-stage build. Node installs dependencies and runs the production build. Only the resulting `dist` directory is copied into the final Caddy image, so the runtime image does not carry the full development toolchain.

Caddy serves static files, enables gzip compression and falls back to `/index.html` when a direct route does not match a physical file. That fallback is what makes URLs such as `/animation` work when opened directly instead of only after navigating from the home page.

No backend still means a server exists

Caddy is a web server, but it is not an application backend. It does not run commission logic, query a database or generate pages. Its job is simply to serve the finished frontend build reliably.

	Previous prototype	Final version
Rendering	Express + EJS	React frontend
Language	JavaScript	TypeScript
Routing	Server routes	React Router
Backend	Express app	None
Runtime	Node application	Caddy serving static build

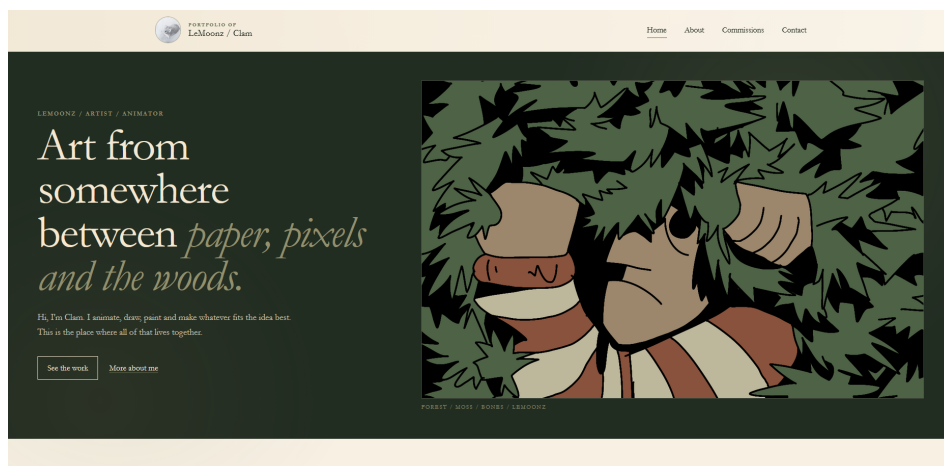
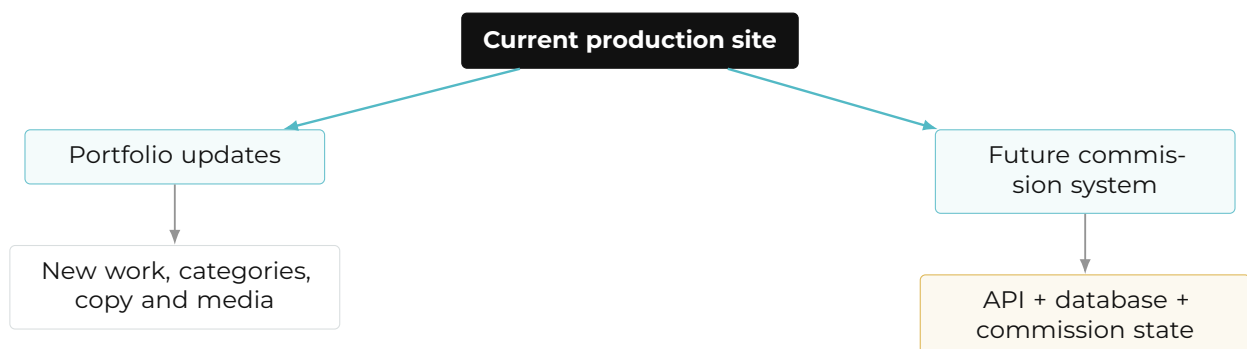
Final state and future development

The production version is complete as a frontend portfolio. The current architecture covers the required content, routing, reusable portfolio presentation, responsive behaviour and deployment.

The final stack is smaller than the original full-stack prototype because the current feature set does not require application-side data processing. React and TypeScript handle the interface and routing, Vite produces the static build, and Caddy serves that build inside the Railway deployment.

The existing structure is ready for normal portfolio growth. New artwork can be added inside the existing category pages, reusable gallery components can render image or video entries, and additional routes can be introduced without changing the deployment model.

A backend becomes relevant if commissions move beyond static information. A future commission system could introduce request submission, persistent order data, availability, status tracking, automated confirmation and an administrative view. Those features would justify an API and database because they introduce state that cannot live only in the frontend.



Project hand-off

The current version can remain a static frontend until a feature creates a real need for persistent server-side state. Thank you to LeMoonz for the trust, feedback and collaboration throughout the project.